

"Verified Abbreviated Zeta 256 bits (VAZ256™): Post-Quantum Digital Signature Scheme with 32-byte Keys, Based on Dilithium5 and SHAKE-256"

Author: Fran Luis Vazquez Alonso

Date: June 2026

Version: 0.2.0

Repository: <https://www.github.com/GrayLupus/vaz256>

License: GPL-3.0

Abstract

VAZ256™ is a post-quantum digital signature scheme that combines Dilithium5 and SHAKE-256 to achieve quantum-resistant signatures using ultra-compact 32-byte public and private keys without compromising security. This scheme aims to provide an efficient alternative for key generation, signing, and verification, with security equivalent to NIST Level 5, making it ideal for systems where key storage is critical. Results show a reduction of over 98% in key size compared to Dilithium5 while maintaining efficient key generation and verification times, surpassing schemes like Falcon1024 in performance.

Keywords: cryptography, post-quantum, digital signature, dilithium, hash

Introduction

Post-quantum cryptography faces the challenge of balancing security, key size, and efficiency. Schemes like Falcon (small keys but complex implementation), Dilithium (efficient but with large public keys), or SPHINCS+ (inefficient in signature size) limit their adoption in storage-constrained systems like blockchain. Verified Abbreviated Zeta 256 bits (VAZ256™) emerges as a hybrid solution that compresses public keys using SHAKE-256 and employs Dilithium5-based signatures, maintaining NIST Level 5 security. Its design enables memorable keys (32 bytes) and fast verification, ideal for systems prioritizing security and efficiency, or networks with optimization mechanisms like signature pruning.

VAZ256 addresses the challenge of reducing public key size through an innovative approach combining key compression with dual-component signatures, achieving an optimal balance between security and performance. The objectives of this work include explaining the logic and foundations of this new scheme and demonstrating its viability through performance comparisons with established post-quantum schemes.

Theoretical Foundations

The core idea behind VAZ256 was to implement it in pure Rust without external dependencies. The code and logic of VAZ256 were developed by modifying the reference implementation of Dilithium5 in CRYSTALS-Dilithium from Quantum-Blockchains (available on GitHub).

Since VAZ256 is a modification of Dilithium5, it is necessary to briefly explain the logic and operation of Dilithium5 before delving into VAZ256 in greater depth.

*For a deeper understanding of Dilithium's implementation and requirements, refer to: <https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf>.

1. Theoretical Foundations and Operation of Dilithium5

1.1. Mathematical Structure

Dilithium5 is a lattice-based digital signature scheme relying on two fundamental computational problems:

- Module Learning With Errors (MLWE):** Given a matrix $A \in R_q^{k \times l}$ and $t = As_1 + s_2$, it is hard to recover s_1, s_2 .
- Module Short Integer Solution (MSIS):** Given A , it is hard to find a small z such that $Az = 0$.

These problems are quantum-resistant and define Dilithium5's security (NIST Level 5, equivalent to 256-bit security).

1.2. Key Parameters

Dilithium5 operates over a polynomial ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, where:

- $q = 8,380,417$ (23-bit prime, defined as $2^{23} - 2^{13} + 1$)
- $n = 256$ (fixed polynomial degree)
- $(k, l) = (8, 7)$
- $\eta = 2$
- $\gamma_1 = 2^{19}$
- $\tau = 60$
- $\omega = 75$

Key Sizes:

- Public key (pk): 2592 bytes (matrix $A \in R_q^{k \times l}$ and compressed vector t).
- Private key (sk): 4864 bytes (vectors s_1, s_2 , seed for A , and pk).
- Signature (σ): 4595 bytes (commitment vector z , challenge c , and vector h).

1.3. Key Generation

1. **Seed and Public Matrix:** Choose a uniform seed $\rho \leftarrow \{0,1\}^{256}$. Derive matrix $A \in R_q^{k \times l}$ via $\text{ExpandA}(\rho)$ using SHAKE-128.
2. **Secret Vectors:** $s_1 \leftarrow S_\eta^l$ (vector with coefficients in $[-2, 2]$, $\eta = 2$). $s_2 \leftarrow S_\eta^k$ (small noise vector with the same distribution).
3. **Public Key Calculation:** $t = As_1 + s_2 \in R_q^k$. Decomposition: $t = t_1 \cdot 2^{13} + t_0$ (using power2round). Public key $pk = (\rho, t_1)$. Private key $sk = (\rho, s_1, s_2, t_0)$.

1.4. Signing Process

1. **Message Hash:** $\mu = \text{CRH}(\text{CRH}(pk) || M)$ using SHAKE-256.
2. **Mask Generation:** $y \leftarrow S_{\gamma_1-1}^l$ where $\gamma_1 = 2^{19}$.
3. **Commitment:** $w = Ay \in R_q^k$. $w_1 = \text{HighBits}(w)$.
4. **Challenge:** $c = H(\mu || w_1) \in B_\tau$ (polynomial with exactly 60 coefficients ± 1).
5. **Response:** $z = y + cs_1$. Reject if $\|z\|_\infty \geq \gamma_1 - \beta$ ($\beta = 120$).

6. **Hints:** $h = \text{MakeHint}(-ct_0, w - cs_2)$. Reject if the number of 1's in $h > \omega$ ($\omega = 75$).
7. **Signature:** $\sigma = (z, c, h)$.

1.5. Signature Verification

1. **Reconstruction:** $w' = Az - ct_1 \cdot 2^{13}$.
2. **Correction:** $w_1' = \text{UseHint}(h, w')$.
3. **Verification Checks:** $c' = H(\mu || w_1')$. Accept if: $c' == c$, $\|z\|_\infty < \gamma_1 - \beta$, number of 1's in $h \leq \omega$.

2. Theoretical Foundations and Operation of VAZ256

2.1. Mathematical Foundations

VAZ256 is a hybrid scheme integrating two critical components:

1. **Dilithium5:** Post-quantum signature scheme based on lattices, with security relying on MLWE and MSIS problems.
2. **SHAKE-256:** Extendable hash function from the SHA-3 family, resistant to collisions and preimage attacks.

2.2. Key Parameters

- **VAZ256 Private Key:** 32 bytes generated randomly (256 bits).
- **VAZ256 Public Key:** 32 bytes (SHAKE-256 hash of the 2592-byte Dilithium5 public key).
- **VAZ256 Signature:** 7187 bytes (4595 bytes for Dilithium5 signature + 2592 bytes for Dilithium5 public key).
- **Security:** Post-quantum: reduced to solving MLWE and MSIS in Dilithium5 parameters. Classical: resistance to SHAKE-256 collisions and preimage attacks (2^{128} operations for attacks).

2.3. VAZ256 Key Generation

1. **Private Key Generation:** Generate a 256-bit random private key (secret).
2. **Public Key Generation:** Use the VAZ256 private key as a seed to deterministically generate a Dilithium5 key pair via `Dilithium5Keypair`. Compress the 2592-byte Dilithium5 public key to 32 bytes using SHAKE-256. The resulting hash is the VAZ256 public key (`vaz256_pk`).

2.4. VAZ256 Signing Process

- 1. Regenerate Dilithium5 Key Pair:** Use the VAZ256 private key (secret) and `Dilithium5Keypair` to regenerate the Dilithium5 key pair (`dilithium5_sk`, `dilithium5_pk`).
- 2. Compute Dilithium5 Signature:** For a message `M`, compute its hash and apply the Dilithium5 signing process to obtain a 4595-byte signature (`dilithium5_signature`).
- 3. Emit VAZ256 Signature:** The final VAZ256 signature is the tuple:
`signature = (dilithium5_signature, dilithium5_pk)`

2.5. VAZ256 Signature Verification

- 1. Public Key Validation:** Split the VAZ256 signature into its components: Dilithium5 signature (4595 bytes) and Dilithium5 public key (2592 bytes). Compute the SHAKE-256 hash of the Dilithium5 public key. If it matches the VAZ256 public key, proceed to the next step.
- 2. Dilithium5 Signature Verification:** Verify the Dilithium5 signature against the extracted Dilithium5 public key and message `M` using the standard Dilithium5 verification process.

2.6. Mitigated Attacks

- 1. Public Key Substitution:** Impossible without breaking SHAKE-256's preimage resistance.
- 2. Signature Forgery:** Requires solving MLWE/MSIS (Dilithium5) or finding SHAKE-256 collisions, but such collisions must also yield a valid Dilithium5 key pair, reaffirming VAZ256's security equivalence to Dilithium5.
- 3. Memory Leaks:** The VAZ256 private key (seed) is securely erased using zeroization techniques.

3. Performance

3.1. Detailed Comparative Analysis (RockChip RK3562 2.0 GHz)

Table 1: Comparison of post-quantum schemes (256-bit security level)

Scheme	Private Key	Public Key	Signature Size	KeyGen	Sign	Verify
--------	-------------	------------	----------------	--------	------	--------

VAZ256	32 B	32 B	7187 B	1.33 ms	5.00 ms	1.01 ms
Dilithium5	4864 B	2592 B	4595 B	1.30 ms	2.10 ms	0.90 ms
Falcon1024	1281 B	1793 B	1330 B	12.50 ms	17.20 ms	2.30 ms
SPHINCS+	64 B	32 B	17220 B	0.15 ms	15.50 ms	0.30 ms

3.2. Advantages of VAZ256

- **Ultra-Compact Keys:** 32 B vs. 2592 B in Dilithium5, ideal for storage-constrained systems (e.g., IoT chips).
- **Verification Efficiency:** 1.01 ms vs. Falcon1024's 2.30 ms, critical for high-frequency blockchain transactions.
- **Full Quantum Resistance:** Combines lattice security (Dilithium5) and hash functions (SHAKE-256), resistant to known quantum attacks.

3.3. Trade-offs and Limitations

- **Large Signatures:** 7187 bytes may be prohibitive for bandwidth-limited networks or systems requiring full historical signature storage without pruning or Merkle tree optimizations.
- **SHAKE-256 Dependency:** Theoretical advances in SHAKE-256 collisions could weaken key compression, though Dilithium5's security remains intact.

3.4. Key Optimizations

- Constant-time operations for sensitive operations (e.g., keygen, signing, hash comparisons).
- Side-Channel Attack Prevention: the Rust implementation restricts hardware register access and uses zeroization for secure memory cleanup.

Conclusions

1. VAZ256 achieves 32-byte public/private keys without compromising post-quantum security, outperforming Falcon and SPHINCS+ in KeyGen

- efficiency. It redefines the key-signature balance by prioritizing tiny keys (32 B) without excessive signature growth (7 KB vs. SPHINCS+'s 17 KB).
2. The Dilithium5-SHAKE-256 dual scheme enables ultra-fast verification (1.01 ms), ideal for high-frequency blockchain and IoT. The combination of lattices and hash functions provides redundancy against future cryptanalytic advances.
 3. Large signatures (7187 B) are viable in systems with pruning or Merkle tree optimizations to avoid full historical storage.

In summary, VAZ256 is a "practical and secure" extension of Dilithium5 that optimizes public key storage without compromising security. Its security reduces to the robustness of Dilithium5 and SHAKE-256, both standardized and widely analyzed. By integrating self-contained signatures and hash-based compression, it offers a unique balance between efficiency and post-quantum resistance.

References

- Bai, S., et al. (2021). CRYSTALS-Dilithium: Algorithm Specifications and Supporting Documentation (NIST Round 3 Submission). <https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf>
- Ducas, L., et al. (2018). FALCON: Fast-Fourier lattice-based compact signatures over NTRU. <https://falcon-sign.info/falcon.pdf>
- NIST. (2023). Post-Quantum Cryptography Standardization Project. <https://csrc.nist.gov/projects/post-quantum-cryptography>